

# SYSTEM INTEGRATION PRACTICES



## Middleware

Instructor: Huy Nguyen Dang Quang, Msc

Email: [huyndq@duytan.edu.vn](mailto:huyndq@duytan.edu.vn)

REF: Shawn A. Butler, Ph.D

Senior Lecturer, Executive Education Program

Institute for Software Research

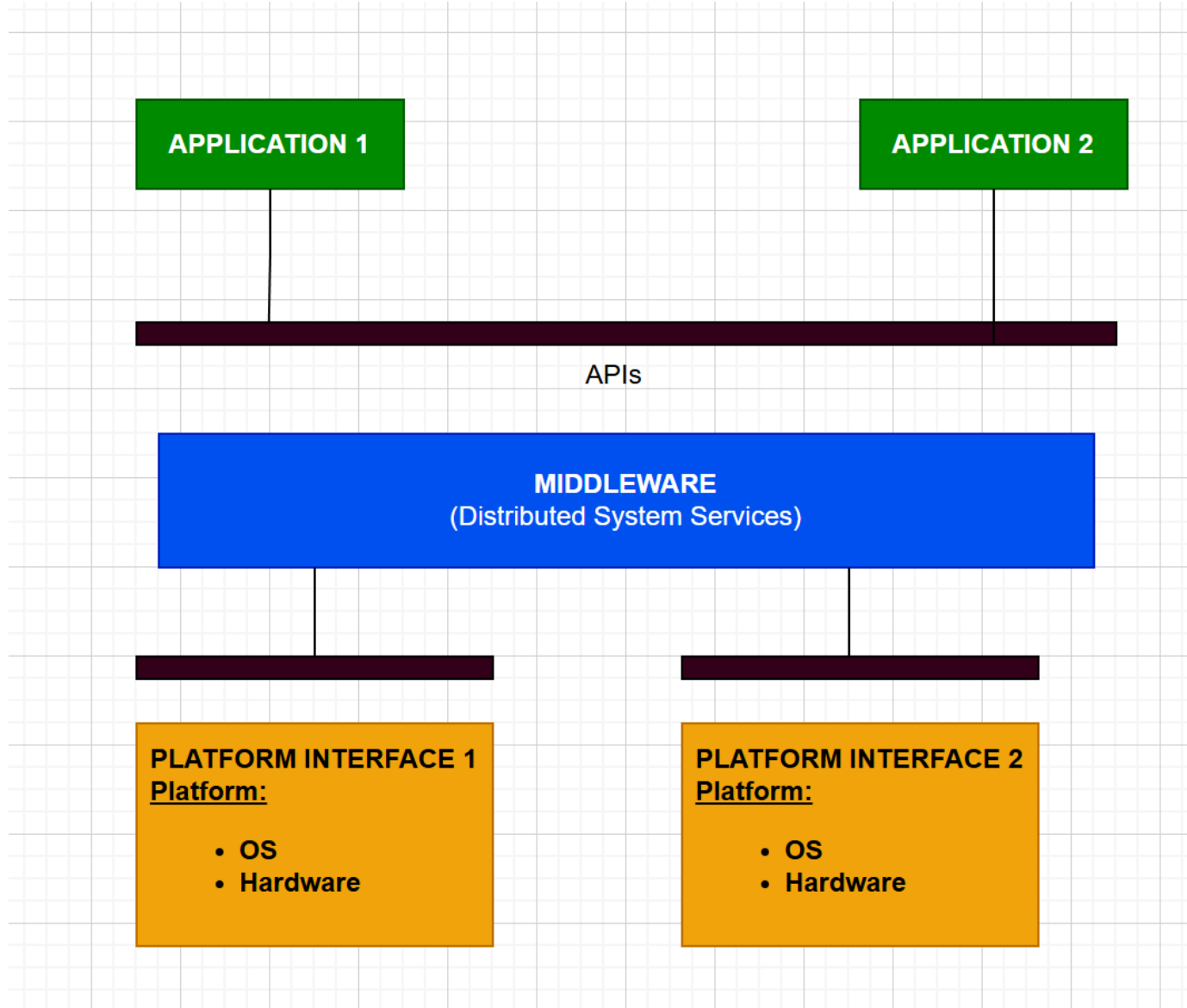
Carnegie Mellon University

- Understand the difference between different types of middleware
  - Message Oriented Middleware
  - Transaction Processing
  - Object Architectures

Middleware is software that acts as an intermediary between applications and systems that simplifies the design, construction, and management of distributed applications. Middleware hides the complexity and diversity of the underlying distributed environment, which includes various networking technologies, computer architectures, and operating systems.

- Modern middleware supports microservices, load balancing, security, and traffic optimization.
- API Gateways like Kong, Apigee, and AWS Gateway manage microservices communication, handling authentication and authorization.

# Middleware?



# Types of Middleware

---

- **Database Middleware:** ODBC, JDBC.
- **Remote Procedure Call (RPC):** Allows remote function calls.
- **Object Request Brokers (ORB):** CORBA, DCOM, JavaBeans.
- **Message-Oriented Middleware (MOM):** MSMQ, RabbitMQ.
- **Transaction-Processing (TP) Monitors:** IBM CICS, Tuxedo.
- **API Middleware:** API Gateway for microservices applications.
- ***Event-Driven Architecture (\*)*:** *Apache Kafka and Amazon Kinesis support event-based systems.*

- **Data Access Middleware:** Communicates between data access programs and databases, such as ODBC and OLE DB.
  - **Example: ODBC (Open Database Connectivity)** - ODBC allows applications to access data from various types of databases without needing to know the specifics of those databases. For instance, an Excel application can use ODBC to retrieve data from a SQL Server database.
- **Remote Procedure Call (RPC) Middleware:** Allows one computer to request services from another computer through remote function calls, typically using LAN networks and requiring high network quality.
  - **Example: gRPC** - A RPC framework developed by Google that enables applications to communicate over a network. For example, a microservice can call a function from another service on a different server through gRPC to retrieve data.
- **Object Middleware (ORB):** Connects applications and software components through standard objects and business logic, such as customers and transactions, with examples like CORBA and DCOM.
  - **Example: CORBA (Common Object Request Broker Architecture)** - CORBA allows applications written in different programming languages to interact with one another. For instance, a Java application can call an object implemented in C++ through CORBA to process a financial transaction.

**Transaction Server Middleware (TP Monitors):** This middleware ensures database transaction properties like Atomicity, Consistency, Isolation, and Durability.

- **Example: IBM's CICS** (Customer Information Control System) - Used for managing transaction processing in mainframe applications, ensuring reliable transaction execution.

**Message-Oriented Middleware (MOM):** This middleware enables processes and applications to exchange information accurately and reliably using a message-based, asynchronous approach.

- **Example: MSMQ (Microsoft Message Queuing)** - Allows applications to send and receive messages between distributed systems in a reliable manner, even if the systems are temporarily offline.



**Event-Driven Architecture (EDA)** is a software architecture where components communicate through events. When an event occurs (e.g., a new order is placed), relevant components respond independently without waiting for each other.

## Key Components:

- 1.Event Producers:** Generate events (e.g., a system creates an event when a new order is placed).
- 2.Event Routers:** Distribute events (e.g., Apache Kafka).
- 3.Event Consumers:** Receive events and perform actions (e.g., update inventory, process payment).

## Example

- In an e-commerce system, a new order event triggers payment processing, inventory updates, shipping coordination, and confirmation emails.

## Advantages

- **Scalability:** Components are independent and easily scalable.
- **Fast response:** The system processes events as they occur.
- **Reduced dependencies:** Components only need to handle events without needing knowledge of each other.
- EDA is ideal for applications requiring real-time responses and scalability, such as e-commerce and IoT.

Messaging is one of the most popular middleware technologies for integrating disparate applications.

- MOM is Middleware that enables communication through messages.
- Model:
  - Point-to-Point (P2P): Each message is sent to a single recipient..
  - Publish/Subscribe (Pub/Sub): Message sent to multiple registered recipients.
- Ex: MSMQ, RabbitMQ, Apache Kafka..

Messaging systems are one of the five types of middleware for distributed applications, focusing on communication and transaction processing.

## Traditional Client/Server Solutions:

1. These use technologies like **RPC** and **ORB** (e.g., **CORBA**, **DCOM**, **Java Beans**), requiring **synchronous communication** where the client waits for a server response.
2. **Example:** A client application calling a service on a server using CORBA.

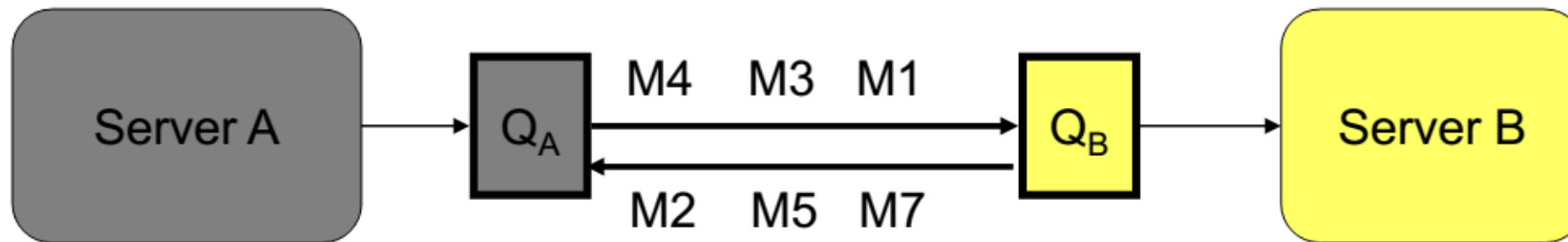
## Distributed Transaction Processing (DTP) Solutions:

1. Examples include **CICS**, **Tuxedo**, and **MTS**, which enable **two-phase distributed transactions** but face integration challenges due to synchronous communication.
2. **Example:** A banking system processing transactions across multiple databases.

## Message-Oriented Middleware (MOM):

1. This allows asynchronous communication, enabling service-requesting components to operate independently of service-providing components.
2. **Example:** **MSMQ** allows different applications to send messages to each other without needing immediate responses, improving system resilience.

# Basic Message Queuing



In request/reply messaging, the request and reply occur within a single transaction

There are two main types of messaging systems: **Point-to-Point Messaging (P2P)** and **Publish-and-Subscribe Messaging (Pub/Sub)**.

## 1. Point-to-Point Messaging (P2P)

- **What It Is:** Each message is sent to one specific recipient (Receiver).
- **How It Works:** Messages are placed in a queue, and the designated receiver picks them up. Once received, the message is removed from the queue.
- **Example:** A customer support request is sent to a queue and handled by the first available support agent.

## 2. Publish-and-Subscribe Messaging (Pub/Sub)

- **What It Is:** Messages are sent to a topic, and multiple subscribers can receive these messages.
- **How It Works:** When a message is sent to a topic, all subscribers automatically receive it.
- **Example:** A news service allows users to subscribe to topics like "Sports" or "Politics." When a new article is published under "Sports," everyone subscribed to that topic gets the article.

## Summary

- **P2P** delivers messages to one recipient only.
- **Pub/Sub** sends messages to multiple recipients who are subscribed to a topic.

When a message is sent through a queue, it must be constructed first and then translated into another format to be usable through translation, transformation, or conversion services. The translation of messages can occur at various points, typically on a server, allowing clients to avoid worrying about a specific message format for each type of queue.

## Points of Message Translation:

### 1.Fixed Point Translation:

1. **Characteristics:** Translation software is distributed to powerful processors.
2. **When to Use:** Best when messages follow a set route.
3. **Example:** In a banking system, transaction messages might be translated at a central server before being sent to other processing servers.

### 2.En-route Translation:

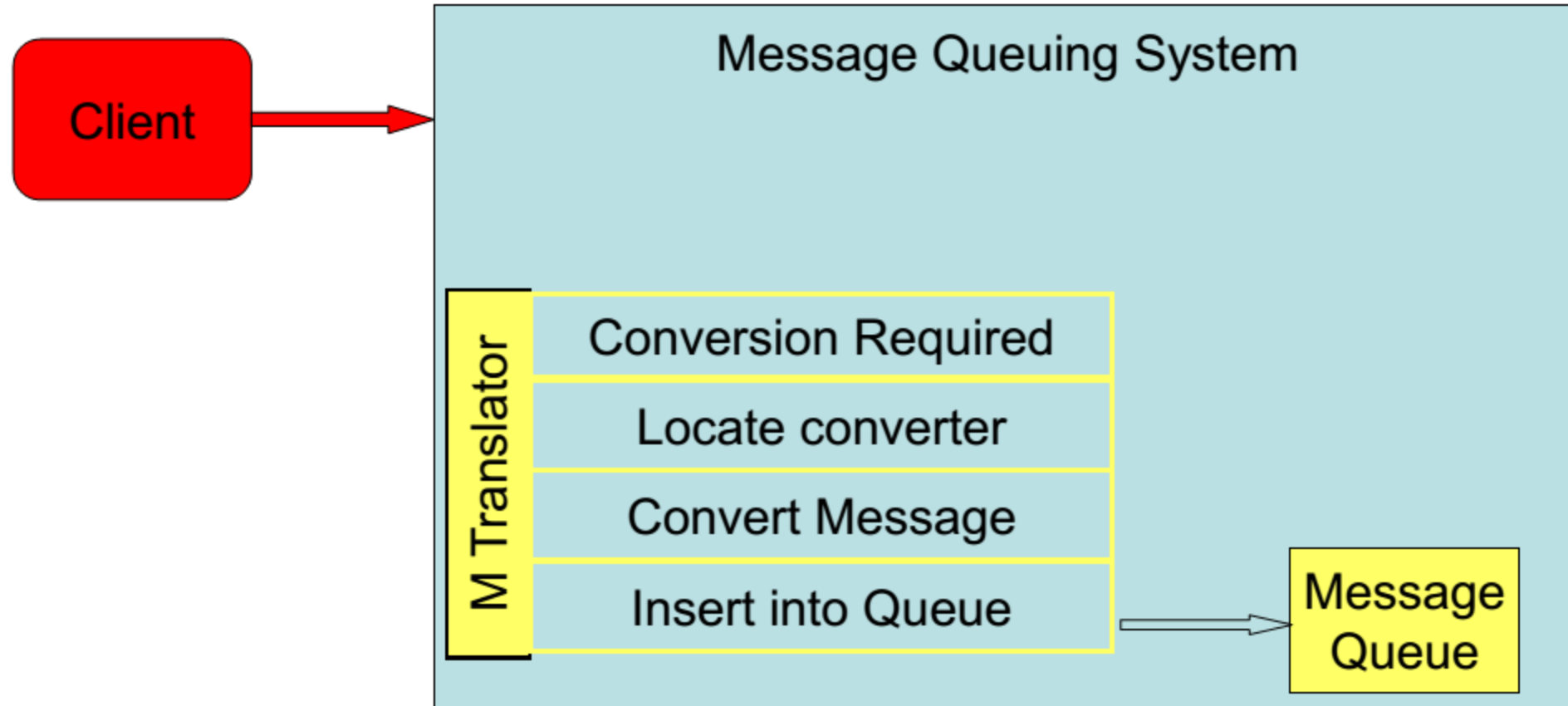
1. **Characteristics:** Takes advantage of spare computing power during transmission.
2. **When to Use:** Routes vary and are more flexible.
3. **Example:** When a mobile application sends a message, the translation could occur on cloud servers between the client and the final server, reducing the load on the mobile device.

## **Conclusion:**

Message translation is a crucial process to ensure that messages can be transmitted and understood between different systems, with two main methods being fixed point translation and en-route translation, each having its own advantages and applications.



# Message Queuing System



When there are multiple queues, managing the entire queuing system becomes very important. Dependencies on a specific list often vary, and queue management must be appropriately capable of supporting these critical dependencies. The dependencies include:

- **Synchronicity:**
  - Definition: The ability to synchronize functions as needed to achieve high performance.
  - Example: In an online banking system, if multiple requests are made to transfer money simultaneously, the system must ensure that these requests are processed in sync to avoid discrepancies in account balances.
- **Response Time:**
  - Definition: The time it takes to reply to a message.
  - Example: In a customer service chatbot, if a user sends a message asking for support, the system should respond within a few seconds to ensure a good user experience.
- **Message Content:**
  - **Definition:** The format and data structures within a message.
  - **Example:** In an e-commerce application, an order confirmation message might include fields for order ID, item details, and total price, formatted in JSON or XML.

## Message Size:

- Definition: The physical size of the message measured in bytes, kilobytes (KB), megabytes (MB), or gigabytes (GB).
- Example: A simple text message might be a few hundred bytes, while a video file attached to a message could be several megabytes.

## Message Priority:

- Definition: The importance of delivering a message, categorized as high, medium, or low priority.
- Example: In a healthcare application, an emergency alert message (high priority) should be processed before routine updates (low priority).

## Queue Volume:

- Definition: The total number of messages present in a queue at any given time.
- Example: During peak shopping seasons, an e-commerce platform might experience a queue volume of thousands of order messages waiting to be processed.

## Queue Timeouts:

- Definition: The maximum duration a queue can hold messages before they are removed or discarded.
- Example: If a message in a queue is not processed within 30 minutes, it might be automatically deleted to free up resources.

## Queue Persistence:

- Definition: The ability to store messages in a persistent form so they are not lost if the system crashes.
- Example: A messaging system that saves messages to a database ensures that even if the server goes down, the messages can be retrieved later.

## Queue Priority:

- Definition: The priority assigned to an entire message queue, determining the order in which messages in that queue are processed relative to others.
- Example: A priority queue for critical system alerts might be processed before regular operational logs to ensure urgent issues are addressed first.

## Conclusion

- Effective queue management is vital in systems with multiple queues, as it ensures that dependencies like synchronicity, response time, and message priority are handled properly to maintain performance and reliability.

- Persistence is the ability to store and retrieve data. Temporary persistence refers to the duration needed for one or more functions to complete, such as a transaction.

# Why Message Persistence?

---

When a message queue is involved in transactions, it's essential to retain messages to ensure reliable delivery and processing. This is particularly important in request/response scenarios where any failure could lead to loss of data or incomplete operations. Here's a breakdown of why message retention is necessary and examples of scenarios that illustrate the need for it:

## 1. Queue Failure:

- **Explanation:** If the message queue system itself fails (due to hardware issues, software crashes, etc.), the messages within that queue may become inaccessible.
- **Example:** Imagine an online payment system where customers send payment requests to a message queue. If that queue crashes and the payment requests are not stored in a persistent database, those requests might be lost, leading to frustrated customers and lost revenue. By storing the messages persistently, the system can recover and process any pending requests after the queue is restored.

# Why Message Persistence?

---

## 2. Request Message is Corrupted:

- **Explanation:** If a request message is corrupted during transmission (e.g., due to network issues), the recipient might not be able to process it correctly.
- **Example:** Consider a scenario where a user submits a form on a website, and the request message gets corrupted (missing fields or incorrect data). If the system retains a copy of the original message, it can identify the corruption and prompt the user to resubmit the correct information, ensuring that the transaction can still be completed.

## 3. Reply Message is Corrupted:

- **Explanation:** If the reply message sent back to the requester is corrupted, the requesting client may not be able to interpret it properly.
- **Example:** In a service where a client asks for the status of their order, if the reply message is corrupted (for instance, garbled text), the client won't receive the correct order status. If the original request was retained, the server could resend the response, ensuring the client gets the accurate information.



# Why Message Persistence?

---

## Recipient Server Fails Before Finishing Process:

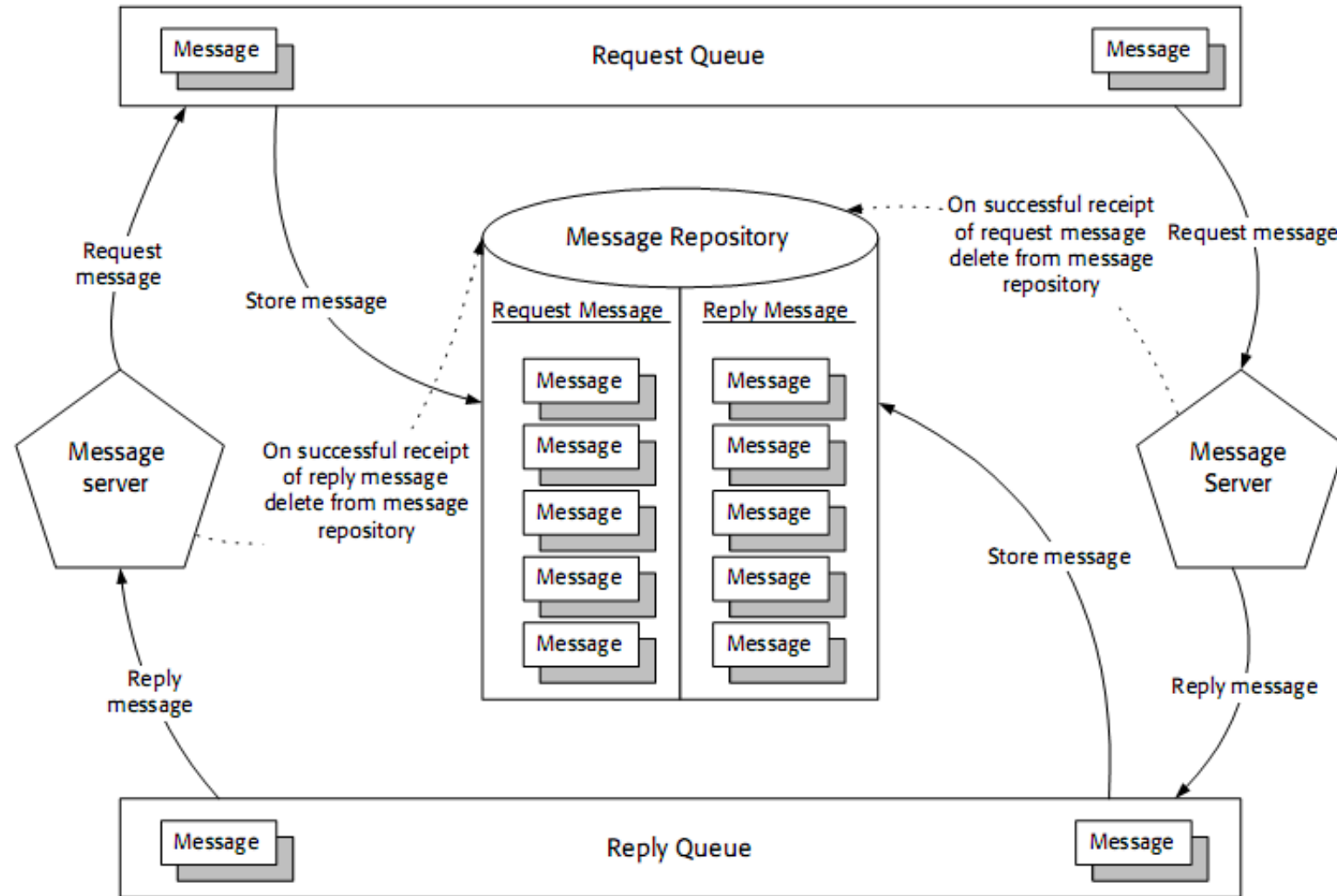
- Explanation: If the server responsible for processing the message crashes or encounters an error before completing the operation, the message could be lost.
- Example: Imagine a booking system where a request to book a flight is processed. If the server fails while processing the booking, and if the original message was not retained, the transaction might not be completed. By storing the message persistently, the system can retry processing it after the server is back online, ensuring that the booking is completed.

## Requesting Client Fails Before Receiving the Reply:

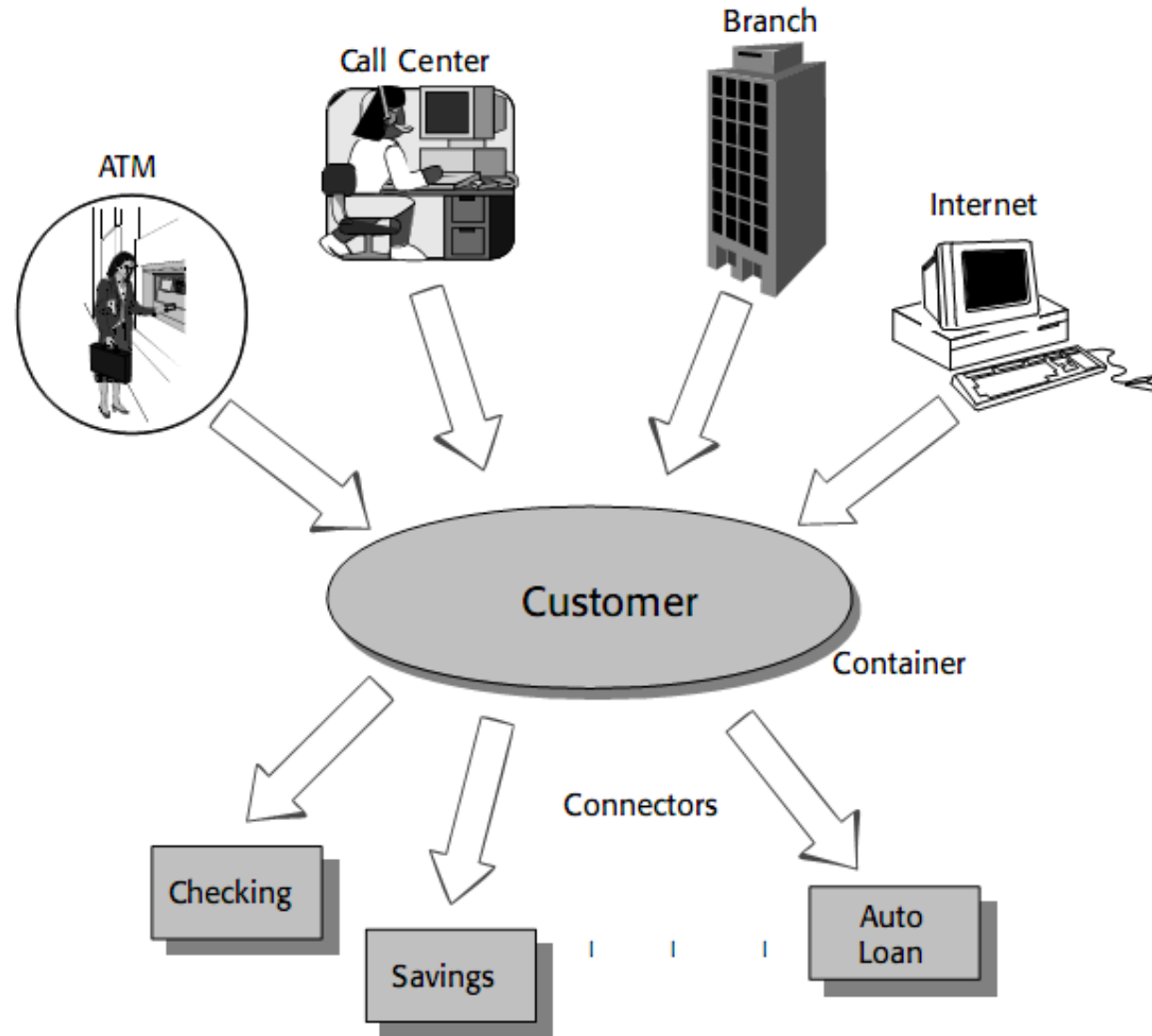
- Explanation: If the client that made the request fails (e.g., due to a network drop or application crash) before it can receive the reply, the client will miss out on important information.
- Example: In a messaging app, if a user sends a message to another user and their device crashes before receiving the acknowledgment, they may think the message was not sent. If the server retains the original request, it can attempt to send the acknowledgment again or store it for retrieval when the client comes back online, ensuring that the user stays informed.

*Retaining messages in a transactional message queue is crucial for ensuring reliable operations, especially in the face of failures. By implementing persistence mechanisms, systems can recover from various issues, safeguarding data integrity and improving user experience.*

# Persistent Queue



# Messaging Example: MegaMoney Bancorp DUY TÂN UNIVERSITY



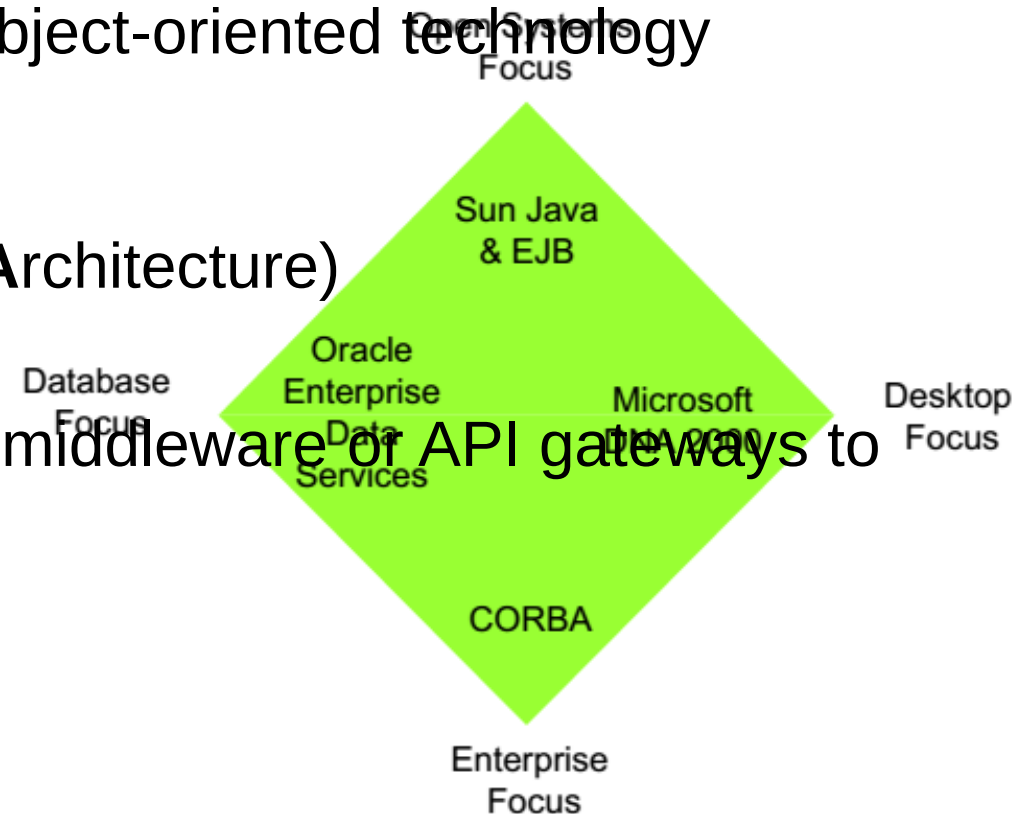
**Figure 4.9** The connector-container model enables centralized access to back-end systems

- A technology architecture will always influence the application architecture in which it is used. When a major technology product, such as an ORB, COM+, or EJB, is employed for product development, it shapes the rest of the architecture around it. The frameworks and services provided by the technology architecture limit the solutions available for building applications using it.
- This is not to say that multiple technologies cannot work together; when they follow different architectural models, system integration technologies must find ways to integrate these architectures with each other.

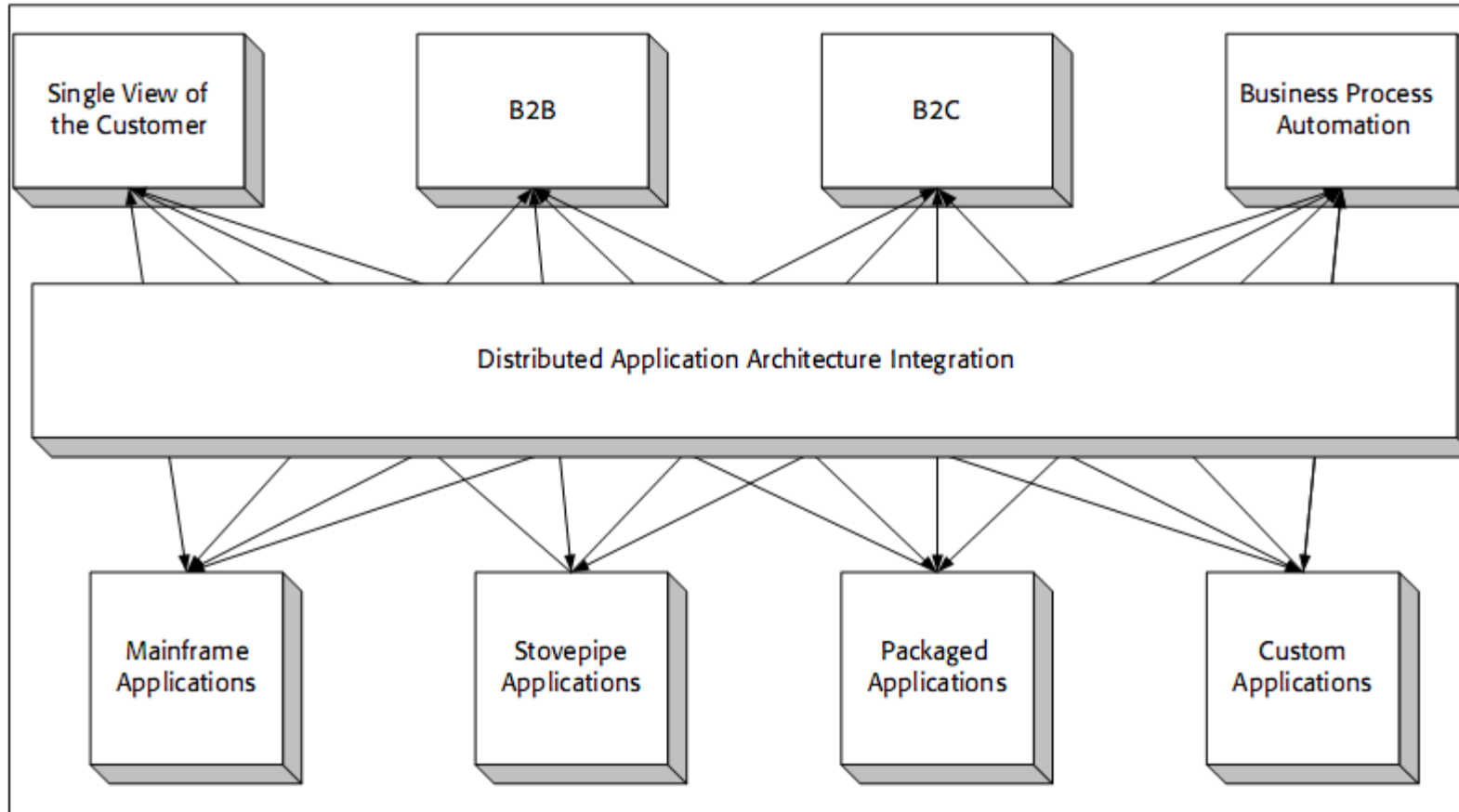
- Technology architecture directly influences how applications are built. When major technologies like **ORB**, **COM+**, or **EJB** are used, they shape the entire application architecture around them. The **frameworks** and services provided by the technology architecture limit the available solutions for developing the application.
- However, multiple technologies can still work together. If they follow **different architectural models**, **system integration technologies** are needed to connect them smoothly.
- **Example:** A company develops an application using **EJB (Enterprise Java Beans)** for the backend and **COM+** for Windows service communication. Since each technology follows its own architecture, they need a **middleware** or **API gateway** to integrate these components into a unified system.

# Commonly Used Object Architectures

- In modern technology architecture approaches, basic components serve as building blocks.
- They are assigned to layers to provide specific types of services and often act as mechanisms for distribution.
- The concept of components directly supports object-oriented technology architectures such as:
  - .Net (formerly DCOM/COM+)
  - CORBA (**C**ommon **O**bject **R**equest **B**roker **A**rchitecture)
  - Enterprise Java Beans
- Different technologies can work together using middleware or API gateways to ensure smooth integration.



# Common Object Architectures



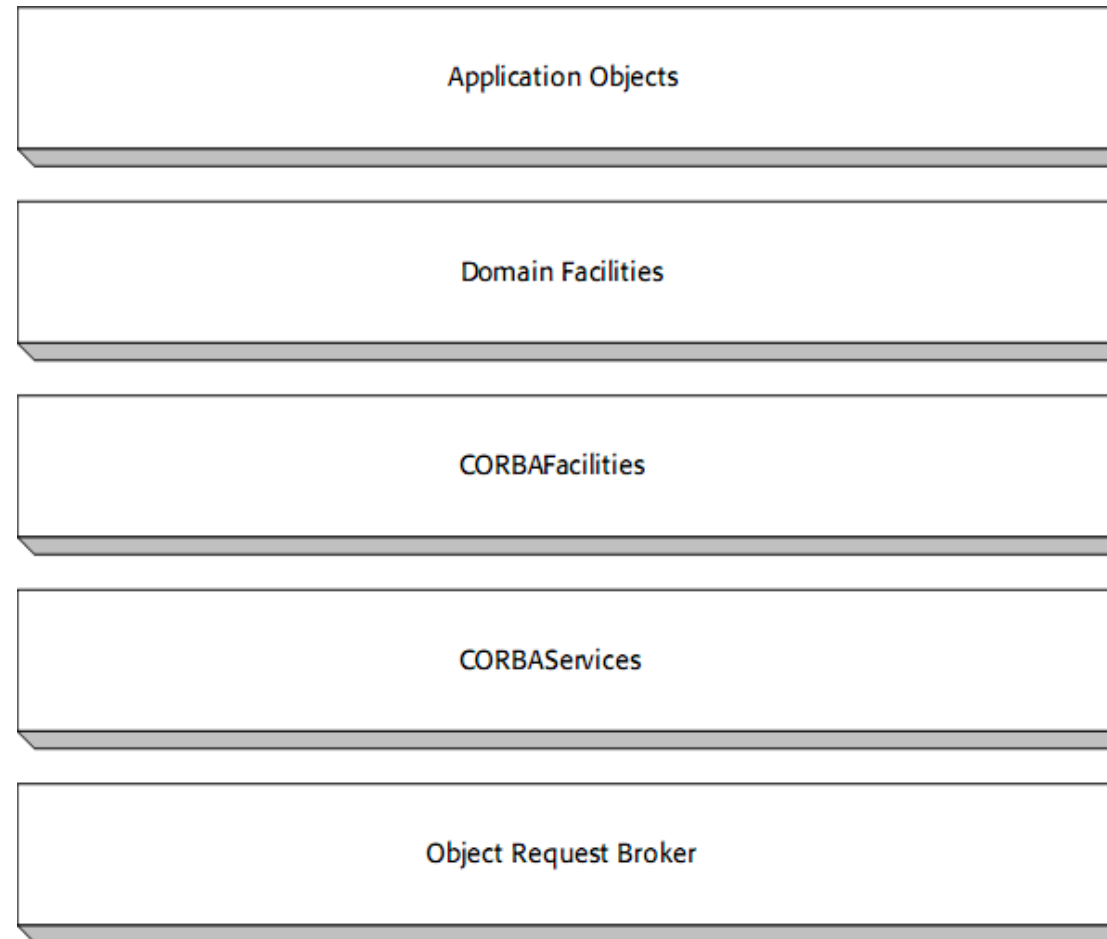
Distributed computing technologies allow for information sharing

- **Object Management Architecture (OMA)** is a framework based on the concept of using an object request broker to facilitate object orientation and distribution (see the figure below).
- OMA supports several different architectural approaches, with the core group of approaches defined by the **Common Object Request Broker Architecture (CORBA)**. CORBA specifies the interfaces used to access CORBA-compliant software.



# Object Management Architecture

---



The Object Management Architecture specifies a CORBA solution to distributed software development and integration.

**CORBA (Common Object Request Broker Architecture)** is a distributed, object-oriented technology that allows the **creation and remote invocation of object methods** across multiple platforms. It enables applications to interact with each other regardless of the operating system or programming language they use.

## **Key Component:**

- **ORB (Object Request Broker):**

A middleware mechanism that transparently forwards client requests to the target object.

## Functions of ORB:

- 1. Operating system transparency** – Works across different operating systems.
- 2. Remote object lifecycle management** – Manages the creation and destruction of remote objects.
- 3. Remote method calls** – Enables method calls on remote objects over a network.

*CORBA allows different systems and applications to communicate and work together seamlessly, no matter what environment or technology they use.*

- CORBA services sit **on top of the ORB** and offer reusable, standard **interfaces** to help developers build distributed applications efficiently. Without these services, developers would need to build them from scratch, increasing **costs** and reducing **reuse, interoperability, and ease of development**.
- **Categories of CORBA Services:**
  1. **Information Management Services:** Manage data (e.g., database access).
  2. **Task Management Services:** Handle events, concurrency, and transactions.
  3. **System Management Services:** Provide naming, lifecycle, licensing, and trader services.
  4. **Infrastructure Services:** Ensure security, time synchronization, and messaging.

# CORBA Facilities and Domains

---

CORBA provides **application-level services** (facilities) and **domain-specific standards** to meet business needs:

- **Application-level services**

- **Distributed Document Facility:** Manages shared documents across systems.
- **Internet Facility:** Supports internet-based communication.
- **Print Facility:** Handles distributed printing.
- **Workflow Management Facility:** Automates business processes.
- **Calendar Facility:** Manages scheduling and events.

- **Domains (business-specific standard)**

- **Electronic commerce**
- **Finance**
- **Medical**
- **Manufacturing**

# **Previous and Current Limitations of CORBA**

---

## **Previous Limitations:**

- No real-time capabilities.
- Lacked robust asynchronous communication.

## **Current Limitation:**

- Only a few essential services are fully implemented.

CORBA offers specialized services and standards for different industries, but it still faces challenges, such as missing real-time features and limited implementation of key services.

# Enterprise Java Beans

**Enterprise Java Beans (EJB)** is a **component-based model** in Java, designed to build **distributed enterprise applications**. EJB handles complex tasks like **transactions, security, concurrency, and database connectivity**, allowing developers to focus on business logic. It also supports **integration with CORBA** to create distributed systems.

|                        |                                      |
|------------------------|--------------------------------------|
| Web Clients            | Java applets, servlets, server pages |
| Database Access        | Java Database Connectivity (JDBC)    |
| Distributed Components | Java Beans<br>Enterprise Java Beans  |
| Distributed Objects    | Remote Method Invocation             |
| Object Directory       | Java Naming and Directory (JNDI)     |
| Transaction Services   | Java Transaction Services (JTS)      |
| Message Service        | Java Message Service (JMS)           |

*This table outlines how Java technologies cover various essential components for **distributed systems**, including web development, database access, transactions, messaging, and object management. Each technology or service plays a role in making Java applications scalable, reliable, and suitable for enterprise use.*

A **container environment** in CORBA handles key services like **transactions, security, and data persistence** while managing **interfaces and event resolution**.

## **Integration with Java Beans (EJB):**

- EJBs can function as **CORBA components**, leveraging CORBA's distributed object capabilities.
- This setup provides a **software distribution format**, making it easier to deploy and integrate applications.

## **Functions of Containers:**

- 1.Track event types:** Monitor different events within the system.
- 2.Provide event channels:** Ensure events are transmitted through the correct paths.
- 3.Host EJB components:** EJBs use containers to interact with CORBA systems.



- **Transaction Processing (TP)** is a system that ensures **ACID properties** (Atomicity, Consistency, Isolation, Durability) are maintained across multiple systems and applications when processing complex transactions.
- **Problem in Distributed Systems:**
  - Booking a **flight, hotel, and rental car** involves **different databases** and **applications** in **separate locations**.
  - If one part fails (e.g., flight not available), it's hard to cancel everything across systems.
- **Limitations of MOM:**
  - MOM (Message-Oriented Middleware) helps systems communicate but can't guarantee ACID.
  - It can lead to partial updates and inconsistent data if any part of the transaction fails.
  - **How TP Systems Help: TP systems manage distributed transactions, ensuring that:**
    - All steps succeed, or
    - Everything rolls back if one part fails.

TP ensures **reliable and consistent** transactions across multiple systems, even when they involve **different locations and delays**.

- Message Oriented Middleware is one of the most popular applications for distributing information among different applications and systems
  - It provide asynchronous delivery of messages among all system components
  - Relies on a naming service to find all components
- Many products are available to support integration of system components
- Transaction Process Monitors (TP Monitors) ensures ACID properties in distributed environments

The end

**THANK YOU**